[redacted]

Mark: 6/20 (total score: 6/20)

+178/1/18+

CCP - Programmation fonctionnelle 24/02/2025

Durée : 60 min. Le seul document autorisé est une feuille A4 blanche manuscrite

[redacted]

1 λ-Calcul

Question 1 (1 pt) Donner un type pour l'expression suivante, ou indiquer que l'expression n'est pas typable. $((\lambda x.\lambda y.(x+y)\ 3)\ 4)$ ☐F ☐J

1/1

$((\lambda x.\lambda y.(\underset{n}{x}+\underset{n}{y})\ 3)\ 4)$ type : Number

Question 2 (1 pt) Donner un type pour l'expression suivante, ou indiquer que l'expression n'est pas typable. $(\lambda x.\lambda y.(x\ y)\ \ \lambda z.(z+5))$ ☐F ☐J

1/1

type : Number → Number

Question 3 (1 pt) Donner un type pour l'expression suivante, ou indiquer que l'expression n'est pas typable. $((\lambda x.\lambda y.((x\ 2)+(y\ 2))\ 3)\ 4)$ ☐F ☐J

0/1

(Number → Number) → Number

Question 4 (1 pt) Donner un type pour l'expression suivante, ou indiquer que l'expression n'est pas typable. $\lambda x.\lambda y.(x\ \lambda z.(y+z))$ ☐F ☐J

0/1

(Number → Number) → (Number → (Number → Number))

Question 5 (1 pt) Donner le résultat de l'évaluation de l'expression suivante de façon à ce que ce résultat ne puisse pas être réduit : $(\lambda x.((x\ 3)+(x\ 2))\ \ \lambda y.(y+2))$ ☐F ☐J

1/1

9

Question 6 (1 pt) Donner le résultat de l'évaluation de l'expression suivante de façon à ce que ce résultat ne puisse pas être réduit : $(\lambda x.\lambda y.(x\ y)\ \ \lambda z.((z\ 3)\ 4))$ ☐F ☐J

0/1

~~(dy.((x 3) y)~~ (u 3)

2 OCaml

2.1 Extraction de paragraphes

On souhaite découper un fichier texte en paragraphes. On dispose pour cela des lignes du fichier sous forme de `string list`, chaque élément de la liste contenant une ligne du fichier.

Chaque paragraphe est séparé du suivant par au moins une ligne vide. Un paragraphe ne peut pas être vide. Pour simplifier, on supposera qu'il n'y a pas d'espace en début ou en fin de ligne. En particulier, une ligne vide est représentée par un `string` vide, il n'y a pas de ligne constituée uniquement d'espaces. Un fichier peut se terminer par un nombre quelconque de lignes vides.

Par exemple on peut considérer le fichier suivant :

```
1
2  Un programme informatique est un ensemble
3  d'instructions et d'opérations destinées à être
4  exécutées par un ordinateur.
5
6  Un programme source est un code écrit
7  par un informaticien dans un langage de programmation.
8  Il peut être compilé vers une forme binaire ou
9  directement interprété.
10
11
12 Un programme binaire décrit les instructions à exécuter
13 par un microprocesseur sous forme numérique. Ces
14 instructions appartiennent à un langage machine.
```

Ce fichier est représenté par une liste de taille 14 dont le premier élément est la chaîne `"Un programme informatique est un ensemble"`. Ce fichier contient 3 paragraphes.

On souhaite écrire une fonction **paragraphes** qui prend la liste des lignes d'un fichier et renvoie la liste des paragraphes du fichier, chaque paragraphe étant représenté par une `string` dans laquelle les lignes sont séparées par des `"\n"`.

Pour cela on va écrire 4 fonctions :

joindre : cette fonction prend une liste de ligne et fabrique un paragraphe en assemblant les lignes entre elles avec un `"\n"` entre chaque ligne.

joindre_tout : cette fonction prend une liste de listes de lignes et transforme chaque liste de ligne en paragraphe (en utilisant la fonction `joindre`). Cette fonction ne doit pas fabriquer de paragraphe vide. Elle ignorera donc les listes de lignes vides.

regroupe : cette fonction prend une liste de lignes et fabrique une liste de listes de lignes, chacune des listes de lignes représentant un paragraphe. Cette fonction peut, si besoin, fabriquer des listes de lignes vides.

paragraphes : prend la liste des lignes d'un fichier et renvoie la liste des paragraphes du fichier.

On rappelle que l'opérateur `^` permet de concaténer deux chaînes de caractères en OCaml. On s'interdira d'utiliser les fonctions du module `List` et on s'interdira également l'utilisation de l'opérateur de concaténation de listes `@`.

Question 7 (2 pt) Donner le code de la fonction `joindre`. On prendra soin de donner le type de chaque argument, ainsi que le type du résultat. ☐F ☐TJ ☐PJ ☐J

0.5/2

```
let rec joindre (l: string list) : string =
  match l with
  | [] -> "\n"
  | x :: l1 -> x :: "\n" :: joindre l1
;;
```

Question 8 (2 pt) Donner le code de la fonction `joindre_tout`. On prendra soin de donner le type de chaque argument, ainsi que le type du résultat. ☐F ☐TJ ☐PJ ☐J

0/2

```
let rec joindre_tout (l: string list) : string =
  match l with
  | [] -> "\n"
  | x :: l1 -> if x = "" then joindre (joindre_tout l1)
               else joindre (x :: joindre_tout l1)
;;
```

Question 9 (3 pt) Donner le code de la fonction `regroupe`. On prendra soin de donner le type de chaque argument, ainsi que le type du résultat. ☐F ☐TJ ☐PJ ☐J

0/3

```
let rec regroupe (l: string list) : string list =
  match l with
  | [] -> []
  |
```

Question 10 (1 pt) Donner le code de la fonction **paragraphes**. On prendra soin de donner le type de chaque argument, ainsi que le type du résultat. ☐F ☐J

0/1

```
let rec paragraphes (l: string list): string list
  match l with
  | [] -> []
  | x :: l1 -> p x :: paragraphe l1)
;;
```

2.2 Arbres additionneurs

On considère une structure d'arbre binaires avec des feuilles et des nœuds internes. Chaque feuille et chaque nœud interne contient un `int`. Il n'y a pas d'arbre vide.

La *valeur* d'un arbre est celle de sa racine (si l'arbre est une feuille, c'est donc la valeur de la feuille). Un arbre est *correct* si sa valeur est la somme de la valeur de ses feuilles. Si tous les sous-arbres d'un arbre sont corrects, alors la valeur d'un nœud interne est la somme des valeurs de ses fils.

Question 11 (2 pt) Définir le type **arbre** qui représente les arbres décrits ci-dessus ☐F ☐J

2/2

```
type arbre
  | Feuille of int
  | Arbre of int * arbre * arbre
;;
```

Question 12 (1 pt) Définir la fonction **valeur** qui donne la valeur d'un arbre. On prendra soin de donner le type de chaque argument, ainsi que le type du résultat. Cette fonction n'est **pas** récursive.

☐F ☐J

0/1

```
let valeur (a : arbre) : int =
  a(x, _, _) -> x
;;
```

Question 13 (3 pt) Définir la fonction `corrige` qui prend un arbre et change les valeurs de ses nœuds internes de façon à ce que chaque sous-arbre du résultat soit *correct*. On prendra soin de donner le type de chaque argument, ainsi que le type du résultat. Cette fonction fera 1 appel récursif par fils.

☐F ☐TJ ☐PJ ☐J

0.5/3

```
let rec corrige (a: arbre): arbre =
  match a with
  | a(x,y,z) → if x = y+z then corrige a(y,-,-)
  }            else corrige (x-z, -, -)
  | a(x,y,z) → if x = y+z then corrige a(z,-,-)
               else corrige (x-y, -, -)
;;
```